

Generalized wording for partial specializations

Document number: P2096R1
Date: 2020-03-23
Project: Programming Language C++, Core Working Group
Reply-to: James Touton <bekenn@gmail.com>

Table of Contents

1. [Table of Contents](#)
2. [Revision History](#)
3. [Introduction](#)
4. [Wording](#)
5. [Acknowledgments](#)
6. [References](#)

Revision History

Revision 1 - 2020-03-23

- Incorporated review feedback from CWG telecon.

Revision 0 - 2020-02-01

- Initial draft.

Introduction

This paper is intended to address CWG issue [1711](#) by generalizing the existing wording for class template partial specializations to allow for partial specializations of variable templates.

Wording

All modifications are presented relative to [N4849](#). "[...]" indicates elided content that is to remain unchanged.

Modify §9.9 [\[namespace.udecl\]](#) paragraph 11:

[*Note*: Partial specializations of `class` templates are found by looking up the primary `class` template and then considering all partial specializations of that template. If a *using-declaration* names a `class` template, partial specializations introduced after the *using-declaration* are effectively visible because the primary template is visible ([\[temp.class.spec\]](#)). — *end note*]

Modify §13.1 [\[temp.pre\]](#) paragraph 4:

A template-declaration can appear only as a namespace scope or class scope declaration. Its *declaration* shall not be an *export-declaration*. In a function template declaration, the last

component of the *declarator-id* shall not be a *template-id*. [*Note:* That last component may be an *identifier*, an *operator-function-id*, a *conversion-function-id*, or a *literal-operator-id*. In a class *or variable* template declaration, if the *class-declared* name is a *simple-template-id*, the declaration declares a *class-template* partial specialization ([[temp.class.spec](#)]). — *end note*]

Modify §13.1 [[temp.pre](#)] paragraph 6:

A template name has linkage ([[basic.link](#)]). Specializations (explicit or implicit) of a template that has internal linkage are distinct from all specializations in other translation units. A template, an *template* explicit specialization ([[temp.expl.spec](#)]), and a *class-template* partial specialization shall not have C linkage. Use of a linkage specification other than "C" or "C++" with any of these constructs is conditionally-supported, with implementation-defined semantics. Template definitions shall obey the one-definition rule ([[basic.def.odr](#)]). [*Note:* Default arguments for function templates and for member functions of class templates are considered definitions for the purpose of template instantiation ([[temp.decls](#)]) and must also obey the one-definition rule. — *end note*]

Modify §13.4.3 [[temp.arg.template](#)] paragraph 1:

A *template-argument* for a template *template-parameter* shall be the name of a class template or an alias template, expressed as *id-expression*. ~~When the *template-argument* names a class template, o~~Only primary *class* templates are considered when matching the template *template-argument* with the corresponding parameter; partial specializations are not considered even if their parameter lists match that of the template *template-parameter*.

Modify §13.4.3 [[temp.arg.template](#)] paragraph 2:

Any partial specializations ([[temp.class.spec](#)]) associated with the primary *class* template ~~or primary variable template~~ are considered when a specialization based on the template *template-parameter* is instantiated. If a specialization is not visible at the point of instantiation, and it would have been selected had it been visible, the program is ill-formed, no diagnostic required. [...]

Modify §13.7 [[temp.decls](#)] paragraph 1:

[*Note:* A *template-id*; (that is, the *template-name* followed by a *template-argument-list*) shall not ~~cannot~~ be specified in the declaration of the *declared name of* a primary template declaration.

[*Example:*

```
template<class T1, class T2, int I> class A<T1, T2, I> { }; // error
template<class T1, int I> void sort<T1, I>(T1 data[I]); // error
```

— *end-example*]

[*Note:* However, this syntax is allowed in *class-template* partial specializations ([[temp.class.spec](#)]). — *end note*]

Rename §13.7.5 [\[temp.class.spec\]](#):

~~Class template partial specializations~~

Modify §13.7.5 [\[temp.class.spec\]](#) paragraph 1:

A ~~primary-class template~~ declaration is one in which the ~~class template name~~ *template-name* is an ~~identifier not followed by a template-argument-list~~. A template declaration in which the ~~class~~ template name is a *simple-template-id* is a *partial specialization* of the ~~class~~ template named in the *simple-template-id*, which shall be a class or variable template. A partial specialization of a ~~class~~ template provides an alternative definition of the template that is used instead of the primary definition when the arguments in a specialization match those given in the partial specialization ([\[temp.class.spec.match\]](#)). The primary template shall be declared before any partial specializations of that template. A partial specialization shall be declared before the first use of a ~~class~~ template specialization that would make use of the partial specialization as the result of an implicit or explicit instantiation in every translation unit in which such a use occurs; no diagnostic is required.

Modify §13.7.5 [\[temp.class.spec\]](#) paragraph 2:

~~Each class template partial specialization is a distinct template and definitions shall be provided for the members of a template partial specialization ([temp.class.spec.mfunc]).~~

Modify §13.7.5 [\[temp.class.spec\]](#) paragraph 4:

~~A class template~~ partial specialization may be constrained (~~[temp.pre]~~[\[temp.constr\]](#)). [...]

Modify §13.7.5 [\[temp.class.spec\]](#) paragraph 5:

The template parameters of a template are specified in the angle bracket enclosed list that immediately follows the keyword `template`. For partial specializations, the template argument list is explicitly written immediately following the ~~class~~ template name. For primary templates, this list is implicitly described by the template parameter list. Specifically, the order of the template arguments is the sequence in which they appear in the template parameter list. For each template parameter P_i , there is a corresponding template argument A_i consisting of the type-id or id-expression naming P_i . [Example: The template argument list for the primary template in the example above is `<T1, T2, I>`. — end example] [Note: The template argument list cannot be specified in the primary template declaration. For example,

```
template<class T1, class T2, int I>
class A<T1, T2, I> { }; // error
```

— end note]

Modify §13.7.5 [\[temp.class.spec\]](#) paragraph 6:

A ~~class template~~ partial specialization may be declared in any scope in which the corresponding primary template may be defined ([\[namespace.memdef\]](#), [\[class.mem\]](#), [\[temp.mem\]](#)). [...]

Modify §13.7.5 [\[temp.class.spec\]](#) paragraph 7:

Partial specialization declarations themselves are not found by name lookup. Rather, when the primary template name is used, any previously-declared partial specializations of the primary template are also considered. [*Note*: One consequence is that a *using-declaration* which refers to a ~~class~~ template does not restrict the set of partial specializations which may be found through the *using-declaration*. — *end note*] [...]

Modify §13.7.5 [\[temp.class.spec\]](#) paragraph 9:

Within the argument list of a ~~class template~~ partial specialization, the following restrictions apply:

- The type of a template parameter corresponding to a specialized non-type argument shall not be dependent on a parameter of the [_partial](#) specialization. [*Example*:

```
template <class T, T t> struct C {};
template <class T> struct C<T, 1>; // error
template< int X, int (*array_ptr)[X] > class A {};
int array[5];
template< int X > class A<X,&array> { }; // error
```

— *end example*]

- The [_partial](#) specialization shall be more specialized than the primary template ([\[temp.class.order\]](#)).
- The template parameter list of a [_partial](#) specialization shall not contain default template argument values.
- An argument shall not contain an unexpanded pack. If an argument is a pack expansion ([\[temp.variadic\]](#)), it shall be the last argument in the template argument list.

Rename §13.7.5.1 [\[temp.class.spec.match\]](#):

Matching of ~~class template~~ partial specializations

Modify §13.7.5.1 [\[temp.class.spec.match\]](#) paragraph 1:

When a ~~class~~ template is used in a context that requires an instantiation of the ~~class~~ [template](#), it is necessary to determine whether the instantiation is to be generated using the primary template or one of the partial specializations. This is done by matching the template arguments of the ~~class~~ template specialization with the template argument lists of the partial specializations.

- If exactly one matching [_partial](#) specialization is found, the instantiation is generated from that [_partial](#) specialization.
- If more than one matching [_partial](#) specialization is found, the partial order rules ([\[temp.class.order\]](#)) are used to determine whether one of the [_partial](#) specializations is more

specialized than the others. If ~~none of the specializations is more specialized than all of the other matching specializations~~ such a partial specialization exists, the instantiation is generated from that partial specialization; otherwise, ~~then~~ the use of the class template is ambiguous and the program is ill-formed.

- If no matches are found, the instantiation is generated from the primary template.

Modify §13.7.5.1 [\[temp.class.spec.match\]](#) paragraph 4:

In a type name that refers to a class template specialization, of a class or variable template (e.g., `A<int, int, 1>`), the argument list shall match the template parameter list of the primary template. The template arguments of a partial specialization are deduced from the arguments of the primary template.

Rename §13.7.5.2 [\[temp.class.order\]](#):

Partial ordering of ~~class template~~ partial specializations

Modify §13.7.5.2 [\[temp.class.order\]](#) paragraph 1:

For two ~~class template~~ partial specializations, the first is *more specialized* than the second if, given the following rewrite to two function templates, the first function template is more specialized than the second according to the ordering rules for function templates ([\[temp.func.order\]](#)):

- Each of the two function templates has the same template parameters and associated constraints ([\[temp.constr.decl\]](#)) as the corresponding partial specialization.
- Each function template has a single function parameter whose type is a class template specialization where the template arguments are the corresponding template parameters from the function template for each template argument in the *template-argument-list* of the *simple-template-id* of the partial specialization.

[...]

Rename §13.7.5.3 [\[temp.class.spec.mfunc\]](#):

Members of class template partial specializations

Modify §13.7.5.3 [\[temp.class.spec.mfunc\]](#) paragraph 1:

The template parameter list of a member of a class template partial specialization shall match the template parameter list of the class template partial specialization. The template argument list of a member of a class template partial specialization shall match the template argument list of the class template partial specialization. ~~A class template partial specialization is a distinct template.~~ The members of the class template partial specialization are unrelated to the members of the primary template. Class template partial specialization members that are used in a way that requires a definition shall be defined; the definitions of members of the primary template are

never used as definitions for members of a class template partial specialization. An explicit specialization of a member of a class template partial specialization is declared in the same way as an explicit specialization of a member of the primary template. [...]

Modify §13.7.8 [\[temp.concept\]](#) paragraph 5:

A concept is not instantiated ([\[temp.spec\]](#)). [Note: A ~~concept-id~~concept-id ([\[temp.names\]](#)) is evaluated as an expression. A concept cannot be explicitly instantiated ([\[temp.explicit\]](#)), explicitly specialized ([\[temp.expl.spec\]](#)), or partially specialized ([\[temp.class.spec\]](#)). — end note]

Modify §13.8.2.1 [\[temp.dep.type\]](#) paragraph 1:

A name refers to the *current instantiation* if it is

- in the definition of a class template, a nested class of a class template, a member of a class template, or a member of a nested class of a class template, the ~~injected-class-name~~injected-class-name ([\[class.pre\]](#)) of the class template or nested class,
- in the definition of a primary class template or a member of a primary class template, the name of the class template followed by the template argument list of the primary template (as described below) enclosed in <> (or an equivalent template alias specialization),
- in the definition of a nested class of a class template, the name of the nested class referenced as a member of the current instantiation, or
- in the definition of a class template partial specialization or a member of a class template partial specialization, the name of the class template followed by the template argument list of the partial specialization enclosed in <> (or an equivalent template alias specialization). If the n^{th} template parameter is a template parameter pack, the n^{th} template argument is a pack expansion ([\[temp.variadic\]](#)) whose pattern is the name of the template parameter pack.

Optional: Rename the following stable names as shown:

Old Name	New Name
temp.class.spec	temp.spec.partial
temp.class.spec.match	temp.spec.partial.match
temp.class.order	temp.spec.partial.order
temp.class.spec.mfunc	temp.spec.partial.member

Acknowledgments

Thanks to Daniel Krüger and Davis Herring for their feedback on an early version of this paper.

References

1. CWG issue 1711: <http://wg21.link/cwg1711>